UNIVERSIDAD CENFOTEC

Curso: Estructuras de Datos

Aplicación de Pilas en el Análisis de Expresiones Aritméticas

Estudiante:

Brandon Jose Urroz Mayen

2026

Introducción y Justificación

Evaluar una operación matemática escrita en notación infija (como *(a + b) * c*) no es directo para una computadora. El sistema debe respetar el orden jerárquico de las operaciones (potencias, multiplicación/división y suma/resta) y la agrupación por paréntesis. Para resolver este desafío, la estructura de datos ideal es la **pila (stack)**, la cual opera bajo el principio LIFO (Last-In, First-Out: el último elemento en entrar es el primero en salir). Esta propiedad la hace perfecta para almacenar temporalmente operadores y operandos y procesarlos en el orden correcto.
A continuación se detallan las operaciones fundamentales de una pila y cómo se aplican en la evaluación de expresiones.

Representación Gráfica de Operaciones Fundamentales de una Pila

A continuación se ilustra de forma visual el comportamiento LIFO (Last-In, First-Out) mediante el estado de una pila de celdas apiladas durante la evaluación de la expresión *a + b*:

Operación	Representación Visual (Pila)	Elemento Afectado	Interacción en Expresiones
Push (Apilar)	[b] ← Cima [a]	Inserta **b** sobre el tope	Guarda temporalmente operandos u operadores pendientes de resolver.
Peek (Ver Cima)	[b] ← Inspección [a]	Consulta **b** sin removerlo	Compara la precedencia del operador del tope con el nuevo token sin alterar la pila.
Pop (Desapilar)	[a] ← Nueva Cima	Extrae **b** de la cima	Obtiene operandos para evaluar un operador o vacía operadores acumulados a la salida.

El Algoritmo Shunting Yard de Dijkstra

- **Control de la prioridad de los operadores:** Permite conservar temporalmente los operadores que deben esperar a que se calculen subexpresiones de mayor precedencia o agrupadas entre paréntesis.
- **Conversión a Notación Postfija (RPN):** A través del algoritmo Shunting Yard de Dijkstra, la pila transforma la expresión infija a postfija, eliminando la necesidad de paréntesis y permitiendo una evaluación posterior en tiempo lineal O(n).

Manejo de paréntesis: Cada paréntesis de apertura '(' se apila para delimitar un subnivel de evaluación. Al encontrar un paréntesis de cierre ')', la pila saca y transfiere los operadores acumulados hasta cerrar el bloque correspondiente.

Para realizar la conversión entre notaciones se utiliza un método clásico diseñado por Edsger Dijkstra.

El **algoritmo Shunting Yard**, diseñado por Edsger Dijkstra, es un método clásico utilizado en la ciencia de la computación para transformar expresiones matemáticas en notación infija (la forma habitual con la que trabajamos los humanos, como *(A + B) * C* a notación postfija o Notación Polaca Inversa (RPN, por sus siglas en inglés). Su nombre proviene de la analogía con una estación de maniobras de ferrocarriles ("shunting yard"), donde los vagones (tokens) son desviados temporalmente a vías secundarias (la pila) antes de ensamblar el tren final.

El funcionamiento principal del algoritmo se basa en la lectura lineal de los elementos de la expresión (operandos, operadores y paréntesis) de izquierda a derecha, empleando dos estructuras fundamentales:

- **Una salida (o cola/lista de salida):** Donde se construye la expresión postfija resultante.
- **Una pila (stack) de operadores:** Se utiliza como almacenamiento temporal para retener operadores y paréntesis hasta determinar su momento correcto de salida según la precedencia.

Las reglas operativas del algoritmo se resumen de la siguiente manera:

- **Operandos (números o variables):** Se envían directamente a la salida.
- **Operadores:** Si se lee un operador, se comparan sus propiedades de precedencia y asociatividad con las del operador en el tope de la pila.

Mientras el operador de la pila tenga mayor (o igual) precedencia, este se extrae de la pila y pasa a la salida. Posteriormente, el nuevo operador se apila.

- **Paréntesis de apertura '(':** Se colocan directamente en la pila para marcar el inicio de un subdominio o agrupamiento.
- **Paréntesis de cierre ')':** Provocan la extracción continua de operadores desde la pila hacia la salida hasta encontrar el paréntesis de apertura correspondiente, el cual se descarta junto con el de cierre.
- **Fin de la expresión:** Al concluir la lectura de los tokens, todos los operadores remanentes en la pila se vuelcan en orden hacia la salida.

Gracias a este mecanismo, el algoritmo garantiza la eliminación de paréntesis no ambiguos y respeta rigurosamente el orden de las operaciones en tiempo lineal O(n), facilitando que la computadora evalúe posteriormente la expresión resultante mediante una simple pila de ejecución.

Ejemplo Paso a Paso: Conversión de (A + B) * C a Notación Postfija

Paso	Token Leído	Estado de la Pila	Salida (Notación Postfija)	Acción Realizada
1	(	(	[vacío]	Apilar '('
2	A	(	A	Enviar operando 'A' a la salida
3	+	(, +	A	Apilar operador '+'
4	B	(, +	A B	Enviar operando 'B' a la salida
5	)	[vacío]	A B +	Desapilar '+' hacia la salida y descartar '('
6	*	*	A B +	Apilar operador '*'
7	C	*	A B + C	Enviar operando 'C' a la salida
Fin	-	[vacío]	A B + C *	Vaciar la pila desapilando '*' hacia la salida

Seguridad, Orden y Eficiencia de la Pila

El uso de la pila no solo facilita la transformación de notaciones, sino que también aporta beneficios fundamentales para la ejecución eficiente y robusta de programas:

- **Determinismo y Orden Estricto:** Al procesar la pila con la regla LIFO, la computadora asegura que las operaciones internas dentro de paréntesis o de mayor precedencia se resuelvan exactamente antes que las operaciones externas.
- **Prevención de Errores Desconocidos:** La pila permite identificar inmediatamente fallas sintácticas como desbalance de paréntesis o falta de operandos, evitando que el sistema entre en estados indeterminados o lance excepciones no controladas.
- **Eficiencia en Tiempo O(n):** Dado que cada elemento de la expresión se inserta y se remueve de la pila un número fijo de veces, la complejidad general permanece en tiempo lineal O(n), optimizando el uso de memoria y procesador.

Ejemplo Concreto en Software de Sistema

Un caso de aplicación real dentro de la ingeniería de software es la implementación de analizadores sintácticos (parsers) y evaluadores de expresiones en compiladores e intérpretes de lenguajes como Java, C++ o Visual Basic.

Por ejemplo, en la construcción de una clase evaluadora en Java, se emplean dos pilas principales: una para manipular los operandos (o números) y otra para gestionar los operadores y paréntesis en tránsito. Al iterar linealmente sobre los tokens de la expresión mediante estructuras como *StringTokenizer*, el software consulta y extrae elementos de la pila según la jerarquía establecida (potencias, multiplicación/división, suma/resta).

Este enfoque modular permite convertir cadenas complejas ingresadas por el usuario a notación postfija y luego evaluarlas linealmente de forma ágil, manejando de paso excepciones aritméticas o de sintaxis. Así se demuestra el rol crucial que desempeñan las pilas tanto en compiladores como en calculadoras o entornos académicos.

Referencias Bibliográficas

Algoritmo Shunting Yard - Wikipedia:
https://es.wikipedia.org/wiki/Algoritmo_shunting_yard
Borbón Alpízar, A. (2005). ¿Cómo evaluar expresiones matemáticas en el computador?